

Course Syllabus and Statement of Expectations

CS 101 — Intro to Computer Science — Fall 2026

This document concerns CS 101, which is titled Intro to Computer Science and is offered during Fall 2026, and it is intended to provide, in a single document that may necessarily repeat itself where the subject matter overlaps, a description of the course, the methods by which the work of the course will proceed, the materials students are expected to obtain or access, the dates that students are expected to notice and independently record, and the standards and procedures that will be used when student work, attendance, participation, examinations, assignments, and the final project are considered. No isolated sentence should be interpreted without reference to the remainder of the document, even where the remainder appears to concern another subject, because instructional, administrative, technical, and evaluative matters frequently overlap.

The instructor of record is Dr. Sarah Chen, who is a Professor of Computer Science. Her university email address is chen@cs.university.edu, and her office is located in the Computer Science Building, Room 214B. Students should use email for matters that are private, that concern an individual record, that involve documentation, or that require an appointment, while questions that could reasonably benefit the class should ordinarily be placed on the class question-and-answer platform discussed later in this document. The fact that a message has been sent does not imply that it has been read immediately, and the possibility of a later response should not be used as a reason to postpone beginning an assignment or reading the instructions that accompany it.

Regular office hours are held on Monday from 2:00 PM until 4:00 PM and on Wednesday from 2:00 PM until 4:00 PM in the Computer Science Building, Room 214B. Friday availability is by appointment. These hours may be used to discuss concepts, feedback, study strategies, project scope, or a student's current attempt at a problem. Students who seek help with a program should bring the problem statement, the version of the program they are actually running, any relevant error output, and a description of what they expected to occur. Office hours are not intended to recreate an entire missed class, replace an assigned reading, or convert an unstarted assignment into a last-minute guided exercise.

GENERAL DESCRIPTION, PURPOSE, AND SCOPE

Intro to Computer Science introduces computational thinking through the design, explanation, testing, and revision of programs. Programming in this course is treated as more than typing instructions until a desired output appears. Students will be asked to identify assumptions, translate prose into a sequence of steps, represent information with appropriate values and structures, decide among alternative paths of execution, repeat operations in a controlled way, divide a large task into smaller responsibilities, and determine what evidence supports the claim that a program behaves correctly. Python is the primary implementation language, but the intellectual subject of the course is broader than the syntax of any one language.

Course topics include variables, expressions, state, conditional control flow, loops, arrays or sequence-like structures, functions, decomposition, abstraction, introductory object-oriented programming, basic algorithmic reasoning, testing, debugging, and the communication of technical decisions. Because later material makes use of earlier material without always announcing that it is doing so, a topic that has already been examined should be understood to have entered the continuing vocabulary of the course. A student who can perform an isolated procedure but cannot explain when it is suitable, what assumptions it makes, or how it might fail has not yet demonstrated the full objective associated with that procedure.

By the conclusion of the course, students should be able to translate a written problem into computational steps; use variables, expressions, conditions, loops, collections, and functions appropriately; read a short program and explain its changes of state and resulting output; construct tests for ordinary, boundary, minimal, and unexpected cases; use decomposition and abstraction to improve clarity; explain introductory object-oriented concepts; use Git and GitHub to maintain an understandable history of work; discuss basic time and space implications of algorithmic choices; and communicate technical reasoning in code, prose, and discussion. These outcomes are cumulative and are not assigned to mutually exclusive portions of the semester.

Class activity may include explanation, demonstration, individual attempts, discussion, short in-class checks, assigned programming work, reading, quizzes, examinations, and project development. Students should preserve notes and returned feedback because a comment made on one assignment may describe a standard that remains relevant later. The course will sometimes ask for a result and sometimes ask for the reasoning that supports the result, and students are responsible for recognizing that an answer without the requested explanation is incomplete even when the answer itself happens to be correct.

ANNOUNCEMENTS AND CONTINUING RESPONSIBILITY

Students are expected to monitor official course communications, maintain their own calendar, and verify information rather than relying entirely on memory, a classmate's recollection, or a screenshot that may have lost its context. An absence does not suspend the calendar, and a technical problem discovered shortly before a deadline does not automatically change that deadline. Any material correction or operational change will be communicated through an official channel, but personal reminders remain the responsibility of the student.

MATERIALS, ACCESS, ATTENDANCE, AND RELATED CONDITIONS

Students must have access to **Introduction to Algorithms, commonly identified as CLRS, 4th Edition**, and **Python Crash Course, 3rd Edition**. The first text will be used for selected explanations and reference concerning algorithms and analytical vocabulary, while the second will serve as a practical reference for Python syntax, programming patterns, and examples. The designation of these books as required does not mean that every page will be assigned, nor should a student assume that a page is irrelevant merely because it was not read aloud or reproduced during class. The optional book **Automate the Boring Stuff with Python** may be used by students who want additional applied examples or a second explanation of a topic.

Required software consists of **Python 3.11 or later**, **VS Code, which is available free**, and Git and GitHub. Students are responsible for knowing which version of Python is running, where their files are stored, which file their editor is executing, and whether the work they intend to preserve has actually been committed and, where applicable, pushed. Demonstrations and routine troubleshooting may assume VS Code even if a student elects to use another editor. A program that behaves in an older or materially different environment is not thereby shown to behave in the environment required by the course.

Gradescope is the platform used for homework submission, while **Ed Discussion is the platform used for class questions and answers**. These platforms are not interchangeable. A file located on a student's computer, attached to an unsent message, stored in a repository but not submitted, or selected in a browser without completion of the submission process is not a submitted homework assignment. Students should inspect the uploaded artifact and confirmation rather than infer success from the fact that they intended to submit. Questions on Ed Discussion should have descriptive titles and sufficient context, but they should not expose complete restricted solutions or assessment details that would interfere with the work of others.

Access to books, software, accounts, and a workable development environment should be arranged before it becomes urgent. Students should test Python 3.11 or later, VS Code, Git, GitHub, Gradescope, and Ed Discussion while there is time to resolve an account, installation, file-format, or authentication problem. Waiting until the final portion of an availability window to discover that a password has expired or the wrong file has been edited does not change the distinction between work that was submitted and work that was not submitted.

Attendance is expected because the course requires repeated practice and because explanation, attempted application, feedback, and revision occur in relation to one another. **Three unexcused absences are allowed per semester**. The word "allowed" in this statement means that the first three unexcused absences do not by themselves trigger the separate one-letter-grade consequence described in the next sentence; it does not mean that missed in-class work, explanations, quizzes, participation, or announcements cease to have consequences. **The fourth unexcused absence drops the final grade by one letter grade**.

Tardiness of ten minutes or more counts as one-half of an absence. Consequently, two arrivals that each meet or exceed the ten-minute threshold accumulate to one absence for purposes of the attendance record. A student arriving late should enter with minimal interruption and determine what instructions or activity have already been missed. Students who believe an attendance entry is inaccurate should raise the matter promptly and identify the date and basis for the requested correction, because an attempt to reconstruct an entire semester at its conclusion is less reliable than a timely inquiry.

Participation accounts for 10% of the final grade. Participation is not defined solely as speaking frequently, nor is it automatically secured by physical presence. It may be demonstrated through preparation, attention to exercises, appropriate contribution, a focused question, an explanation of why a loop terminates, the proposal of a useful test case, constructive collaboration within the stated boundaries, or evidence that feedback has been understood and applied. Students should leave room for classmates, distinguish a question from a claim, and revise a claim when the available evidence no longer supports it.

There are no make-ups for missed in-class quizzes without documentation. A student seeking a make-up must therefore provide appropriate documentation; a retrospective statement without documentation does not automatically establish eligibility. Communication should occur as soon as reasonably possible. The existence of documentation permits the request to be considered but should not be read as a guarantee of a particular outcome before the instructor has reviewed the circumstances under applicable university practice.

An absence, whether excused or unexcused, does not create an individualized calendar. Students remain responsible for reading, assignments, announcements, and preparation for the next meeting. Before asking the instructor to reproduce a missed class, a student should consult posted materials and obtain suitable notes. This continuity requirement operates alongside, rather than instead of, the numerical attendance and participation rules described above.

COMMUNICATION IN ORDINARY COURSE OPERATIONS

Email to chen@cs.university.edu is appropriate for private matters, personal records, documentation, and Friday appointment requests. Ed Discussion is appropriate for a question whose answer may assist others. Gradescope is used for homework submission. Monday and Wednesday office hours from 2:00 PM to 4:00 PM in Computer Science Building, Room 214B, as well as Friday meetings by appointment, provide an opportunity for discussion that requires more context than a short message can reasonably supply.

EVALUATION, CATEGORY WEIGHTING, AND INTERPRETATION OF RESULTS

The final grade is computed from four categories whose percentages indicate their contribution to the overall result rather than the amount of time a student may personally spend on them or the number of separate items that may appear in a category. Exams account for 50% of the final grade, assignments account for 30%, participation accounts for 10%, and the final project accounts for 10%. A category containing fewer items may carry greater consequence per item than a category containing many smaller items. Students should therefore not infer importance from frequency alone, and they should not assume that an item described briefly is necessarily worth less than an item described at greater length.

Examinations may ask students to recognize, trace, compare, explain, and apply course concepts under stated conditions. They may combine topics introduced separately, including variables, control flow, loops, functions, collections, decomposition, testing, algorithms, and object-oriented reasoning. A correct output unsupported by a requested method or explanation may receive less credit than a complete response, while an otherwise coherent method containing a limited mechanical mistake may receive partial credit where the work permits the reasoning to be seen. The Midterm Exam and Final Exam are dated elsewhere in this document, and the final examination is cumulative.

Assignments provide sustained practice with implementation, testing, and written reasoning. A program is not complete merely because it ran once for the author. Students should reread the prompt, identify required inputs and outputs, test ordinary and boundary cases, remove irrelevant diagnostic output, use meaningful names, divide responsibilities when division improves clarity, and submit the requested artifacts through Gradescope. The artifact that is submitted is the artifact that can be evaluated. A more recent local version, an unpushed repository change, or a file the student meant to upload cannot be substituted retroactively merely by showing that it existed elsewhere.

The participation category, which accounts for 10% of the final grade, is interpreted with reference to the attendance and engagement language on the preceding page. Participation is distinct from the separate attendance consequence under which the fourth unexcused absence drops the final grade by one letter grade. Thus, a student may lose participation credit through insufficient engagement and may separately incur the stated one-letter-grade consequence through a fourth unexcused absence. These are related course expectations but not duplicate names for the same calculation.

The final project accounts for 10% of the final grade and requires students to integrate several course ideas into a coherent computational artifact. The work includes the selection of a feasible scope, identification of a problem or intended use, consideration of inputs and outputs, implementation, testing, explanation, and final submission. The proposal and final submission have separate dates. A broad idea that remains incomplete or cannot be tested is not automatically stronger than a smaller idea executed carefully and explained accurately.

The letter-grade scale is as follows, expressed here in prose because the boundaries themselves, rather than a visual arrangement, control the interpretation: an A corresponds to 90 through 100; a B corresponds to 80 through 89; a C corresponds to 70 through 79; a D corresponds to 60 through 69; and an F corresponds to a result below 60. After the weighted categories have been computed, the expressly applicable attendance rule must still be considered. As stated previously, the fourth unexcused absence drops the final grade by one letter grade.

Work will be considered with respect to correctness, completeness, clarity, maintainability, testing, reasoning, and adherence to the stated submission requirements as those considerations apply to the particular assessment. Correctness concerns whether the work addresses the actual prompt and produces justified results. Completeness concerns whether all requested parts, files, explanations, or tests are present. Clarity concerns whether another reader can follow the structure and terminology without reverse-engineering unnecessary complexity. Maintainability concerns whether names, decomposition, and comments are proportionate and useful rather than decorative.

Testing should include more than the example that first came to mind. Depending on the problem, useful evidence may include ordinary inputs, boundary values, minimal or empty inputs, repeated values, invalid inputs, or conditions likely to expose an assumption. Students may be asked not only what happened but why it happened and whether the observed result is sufficient evidence for a general claim. Statements such as “it worked on my computer” describe an observation but do not, without further detail, establish the range over which a solution is correct.

Feedback identifies a next step and should be read before a student asks for a score to be changed. A question about evaluation should identify the relevant criterion, the portion of the submitted work that addresses it, and the reason the student believes the recorded evaluation may not match the stated expectation. A request for review is an invitation to re-examine the work, not a promise that the score will increase. Broad statements about effort, desired averages, or the need for a particular grade do not by themselves identify an evaluative error.

The recommended sequence is to read all written and rubric feedback, reread the requirement at issue, locate the submitted evidence that addresses it, and formulate a concise question. Where the matter requires extended context, the student should bring the work to office hours. Because missed in-class quizzes have no make-up without documentation, a request concerning a missed quiz should also include the appropriate documentation rather than treating a general grading question as a substitute for that requirement.

DATED MATTERS, SEQUENCING, AND CALENDAR LANGUAGE

The dates in this section should be entered into a personal calendar with their times, because the date alone is not sufficient to distinguish a morning quiz, an afternoon examination, an end-of-day submission, and a reading that must be completed before class. All stated times refer to the course's local campus time. A date should be understood as the end point of a preparation process rather than as the recommended moment to begin reading a prompt, configuring software, or deciding what a project will do. Personal calendar entries are aids, and a transcription error in a personal calendar does not amend the course information.

The first expressly dated assessment is **Quiz 1 — Variables & Control Flow, which occurs on July 5 at 11:00 AM**. Students preparing for it should be able to reason about values, expressions, assignment, conditions, and execution paths, including the ability to predict behavior before running code. **Assignment 2 Due — Loops & Arrays is due on July 10 at 11:59 PM**. That assignment concerns repeated processing, iteration, sequence-like structures, tracing, and the testing of boundary positions, and its availability before the deadline should not be confused with permission to postpone all work until July 10.

The Midterm Exam occurs on July 14 at 2:00 PM. It is a cumulative checkpoint covering course concepts and methods addressed through the midpoint rather than a boundary after which those concepts disappear. Four days later, **the Project Proposal is due on July 18 at 11:59 PM**. The proposal should describe a defined problem or intended purpose, a feasible scope, the anticipated inputs and outputs, and an initial plan. The proposal is intended to expose ambiguity and excessive scope while revision remains possible, and it should not be treated as a ceremonial title page for a project whose design has not been considered.

Reading — Chapters 8–10 must be completed on July 21 before class. “Before class” means before the relevant class meeting begins, not before midnight and not at some unspecified point later on July 21. Students should read with enough attention to identify unfamiliar terms, relate the material to prior examples, and arrive with questions that can support class activity. The fact that the reading has no clock time expressed numerically does not make it less definite than an item due at 11:59 PM.

Quiz 2 — Functions & OOP occurs on July 25 at 11:00 AM. Its subject matter includes functions, interfaces, decomposition, objects, responsibilities, and introductory object-oriented design reasoning. As with all missed in-class quizzes, there is no make-up without documentation. Students should not wait for the quiz to discover that they can recognize vocabulary but cannot explain how a function's inputs, result, side effects, or relationship to another part of a program affect the design.

The Final Project must be submitted on July 28 at 11:59 PM. Submission means that the complete artifact and any specifically requested supporting material have been placed in the required submission location and verified. Students should run the version they intend to submit, confirm that all required files are present, and avoid relying on local state that the evaluator cannot access. **The Final Exam begins on July 31 at 9:00 AM** and is cumulative. It may require students to connect ideas that were first introduced at different points in the course.

The proximity of the final three dated obligations is intentional calendar information that students must notice: Quiz 2 takes place on July 25 at 11:00 AM, the Final Project is due on July 28 at 11:59 PM, and the Final Exam begins on July 31 at 9:00 AM. These items occur within a seven-day period. Project work should therefore proceed before the quiz, and preparation for the final examination should not be deferred until after the final project has been submitted. The existence of multiple obligations is a reason to plan backward, not a basis for assuming that one of the published times is approximate.

For avoidance of ambiguity, the complete set of eight dated items is repeated in continuous prose, even though several have already been described: Quiz 1 — Variables & Control Flow is July 5 at 11:00 AM; Assignment 2 Due — Loops & Arrays is July 10 at 11:59 PM; the Midterm Exam is July 14 at 2:00 PM; the Project Proposal is due July 18 at 11:59 PM; Reading — Chapters 8–10 is due July 21 before class; Quiz 2 — Functions & OOP is July 25 at 11:00 AM; the Final Project is submitted July 28 at 11:59 PM; and the Final Exam is July 31 at 9:00 AM. This repetition does not excuse a student from recording the information in a usable calendar.

Gradescope should be checked after homework submission because local timestamps do not replace a platform record. Readings should be completed before the class for which they are assigned because later completion cannot retroactively supply preparation. Examinations and in-class quizzes require timely presence. Project milestones require attention to both the technical artifact and the requested supporting explanation. Each of these statements is an application of the same general principle: the relevant action must be completed in the relevant form by the relevant time.

RELATIONSHIP BETWEEN DATES AND EVALUATION

The calendar does not alter the category weights. Exams still account for 50% of the final grade, assignments for 30%, participation for 10%, and the final project for 10%. Likewise, the appearance of quizzes among the dated items does not alter the separate statement that there are no make-ups for missed in-class quizzes without documentation. Students should read the calendar together with the evaluation and attendance language rather than attempting to infer all applicable rules from the date paragraph alone.

FURTHER POLICIES, PROJECT EXPECTATIONS, AND CONCLUDING NOTICE

The final project, which accounts for 10% of the final grade, is intended to show that a student can connect problem definition, scope, representation, control flow, decomposition, implementation, testing, and explanation. The Project Proposal due July 18 at 11:59 PM should make the intended problem and limits understandable before the student has invested all available time in implementation. The Final Project submitted July 28 at 11:59 PM should include the complete artifact and the supporting material specifically requested. A project should not become larger merely to appear more impressive if the additional size prevents the result from being completed, tested, or explained.

Learning computer science commonly involves documentation, examples, discussion, tools, feedback, and assistance, but access to those resources does not remove the obligation to submit work that complies with the instructions and that the student understands. Unless an assessment expressly states otherwise, students should assume that submitted reasoning and implementation must be their own and that they may be asked to explain any portion of it. General concepts and debugging strategies may be discussed within announced boundaries, but complete restricted solutions should not be exchanged, copied work should not be disguised, another person's account should not be used, and retrieved or generated material should not be represented as original understanding when such use is not permitted.

Questions about permitted assistance should be asked before submission. A question asked in advance can clarify a boundary, while a question asked after prohibited material has been incorporated into submitted work may not reverse the underlying action. Version history, source acknowledgment where required, and the ability to explain a decision may be considered when authorship or process is relevant. Students should preserve an interpretable Git and GitHub history instead of relying on a single last-minute upload whose relationship to the development process cannot be understood.

Students who need disability-related accommodations should use the university's established accessibility process and communicate approved arrangements with sufficient time for implementation. Students are not required to disclose unnecessary medical details to classmates. The course should remain respectful of different backgrounds, levels of prior programming exposure, communication styles, and learning paths. Introductory does not mean effortless, and prior experience does not eliminate the need to follow course conventions or permit a student to prevent others from participating.

Students without prior programming experience should begin early, practice regularly, retain examples and feedback, and use office hours before confusion compounds. Students with substantial prior experience should still explain their work at the level requested, use the required environment, and demonstrate that they understand the course's terminology and standards. In both cases, the relevant question is not whether the student has seen a topic before but whether the submitted and observed work demonstrates the stated learning objective under the conditions of this course.

Files should be backed up in more than one recoverable location. Git and GitHub are required tools and should be used to create a meaningful history. Python 3.11 or later is the required interpreter. VS Code is the recommended free editor. Gradescope is used for homework submission, and Ed Discussion is used for class questions and answers. A student should understand where a file is saved, which version is being executed, what has been committed, what has been pushed, what has been uploaded, and what the submission platform confirms. These are separate states and should not be collapsed into the statement that the work is "on the computer."

The instructor may clarify language, correct an error, or adjust operations when circumstances require. A material change will be communicated through an official course channel. Students should distinguish a clarification, which explains how an existing expectation applies, from a change, which alters an expectation. Personal notes, reminders, screenshots, and calendar entries remain useful but do not supersede an officially communicated correction. Students remain responsible for monitoring those communications even when they believe they have already copied every date from an earlier version.

For purposes of consolidating information without replacing the fuller language elsewhere, students should understand the following facts as continuing throughout the course: CS 101 is Intro to Computer Science in Fall 2026; the instructor is Dr. Sarah Chen, Professor of Computer Science, at chen@cs.university.edu and Computer Science Building, Room 214B; office hours are Monday and Wednesday from 2:00 PM to 4:00 PM, with Friday available by appointment; the required books are Introduction to Algorithms (CLRS), 4th Edition, and Python Crash Course, 3rd Edition; the required software and services include Python 3.11 or later, VS Code, Git and GitHub, Gradescope, and Ed Discussion; and Automate the Boring Stuff with Python is optional.

The same consolidation should also preserve, without simplifying away their interaction, that three unexcused absences are allowed per semester, the fourth unexcused absence drops the final grade by one letter grade, tardiness of ten minutes or more counts as half an absence, participation accounts for 10% of the final grade, and missed in-class quizzes have no make-up without documentation. Exams account for 50% of the final grade, assignments for 30%, participation for 10%, and the final project for 10%. An A is 90–100, a B is 80–89, a C is 70–79, a D is 60–69, and an F is below 60.

Reading this concluding language is not a substitute for reading the preceding pages, because the preceding pages describe how the facts operate and how students are expected to act on them. By remaining enrolled and participating in the course, a student is expected to consult the syllabus, monitor official communication, seek clarification before an avoidable problem becomes urgent, preserve appropriate records, and complete work in the stated form by the stated time.